6. Bullshark 컨센서스 프로토콜

6.1 Bullshark 프로토콜 개요

Bullshark는 Tusk의 개선된 버전으로, 부분 동기 환경에서 더 나은 성능을 제공하는 컨센서스 프로토콜입니다.

6.1.1 Tusk 대비 개선사항

- **더 빠른 커밋**: 2 라운드 단축 (r-2 → r)
- **향상된 활성**: 부분 동기 환경에서 더 빠른 진행
- **단순화된 로직**: 복잡성 감소로 구현 용이성 향상
- **동일한 안전성**: Tusk와 동일한 안전성 보장

6.1.2 프로토콜 구조

```
pub struct Bullshark {
pub committee: Committee,
pub store: Arc<ConsensusStore>,
pub gc_depth: Round,
}
```

6.2 핵심 차이점 분석

6.2.1 리더 커밋 조건 비교

```
// Tusk: r-2 라운드의 리더를 r 라운드에서 커밋
let leader_round = r - 2; // Tusk

// Bullshark: r 라운드의 리더를 즉시 커밋
let leader_round = r; // Bullshark
```

6.2.2 지지 확인 방식

```
impl Bullshark {
fn process_certificate(
&mut self,
state: &mut ConsensusState,
consensus_index: SequenceNumber,
certificate: Certificate,
) -> StoreResult<Vec<ConsensusOutput>> {
let round = certificate.round();
let r = round - 1;
// 짝수 라운드에서만 리더 선출 (Tusk와 동일)
if r % 2 != 0 || r < 2 { // Bullshark는 r < 2 (vs Tusk r < 4)
return Ok(Vec::new());
}
let leader_round = r; // 핵심 차이점: 즉시 커밋
let (leader_digest, leader) = match Self::leader(&self.committee, leader_round,
```

```
Some(x) => x,

None => return Ok(Vec::new()),

};

// 현재 라운드(round)에서 리더에 대한 지지 확인

let  stake: Stake = state

.dag

.get(&round) // Tusk는 &(r-1), Bullshark는 &round

.expect("We should have the whole history by now")

.values()

.filter(|(_, x)| x.header.parents.contains(leader_digest))

.map(|(_, x)| self.committee.stake(&x.origin()))

.sum();

if  stake < self.committee.validity_threshold() {

return Ok(Vec::new());

}

// 리더 체인 커밋 (Tusk와 동일한 로직)

self.commit_leader_chain(state, leader, consensus_index)

}

}
```

6.3 성능 향상 메커니즘

6.3.1 지연시간 단축

- **Tusk**: 4 라운드 지연 (리더 생성 → 지지 수집 → 확인 → 커밋)
- **Bullshark**: 2 라운드 지연 (리더 생성 → 즉시 커밋)

6.3.2 처리량 개선

```
// 더 빠른 커밋으로 인한 처리량 향상
pub struct BullsharkMetrics {
commit_latency: Histogram, // 평균 2 라운드
throughput: Counter, // Tusk 대비 ~2배 향상
leader_utilization: Gauge, // 리더 활용도 증가
}
```

6.4 안전성 분석

6.4.1 안전성 보장

Bullshark는 Tusk와 동일한 안전성 속성을 유지합니다:

```
// 안전성 불변조건
fn safety_invariant(&self, state: &ConsensusState) -> bool {
// 1. 모든 커밋된 인증서는 유효함
for cert in &state.committed_certificates {
if !self.verify_certificate(cert) {
return false;
```

```
}
}
// 2. 커밋 순서는 일관성 있음
if !self.verify_commit_order(&state.commit_sequence) {
return false;
}
// 3. 비잔틴 노드가 임의 삽입할 수 없음
if !self.verify_byzantine_resistance(state) {
return false;
}
true
}
```

6.4.2 활성 개선

```
impl Bullshark {
// 부분 동기 환경에서의 활성 보장
fn  ensure_liveness_partial_sync(&self,
state: &ConsensusState,
round: Round
) -> bool {
// GST(Global Stabilization Time) 이후 보장
let  gst_passed = self.check_gst_condition();
```

```
let sufficient_certificates = self.check_certificate_threshold(state, round);
gst_passed && sufficient_certificates
}
}
```

6.5 리더 선출 최적화

6.5.1 동일한 리더 선출 로직

```
impl Bullshark {
fn leader<'a>(
committee: &Committee,
round: Round,
dag: &'a Dag,
) -> Option<&'a (CertificateDigest, Certificate)> {
// Tusk와 동일한 스테이크 가중 선출
cfg_if::cfg_if! {
if #[cfg(test)] {
let leader = committee.authorities.iter().next()
.expect("Empty authorities table!").0;
} else {
let leader = &committee.leader(round);
}
```

```
}
dag.get(&round).and_then(|x| x.get(leader))
}
}
```

6.5.2 리더 효율성 향상

- **즉시 활용**: 선출된 리더를 바로 활용
- **대기시간 단축**: 추가 라운드 대기 불필요
- **처리량 증대**: 더 많은 리더가 활용됨

6.6 구현 복잡성 비교

6.6.1 코드 복잡성

```
// Bullshark가 Tusk보다 단순한 조건
// Tusk
if  r % 2 != 0 || r < 4 {
return Ok(Vec::new());
}
let  leader_round = r - 2;
let  support_round = r - 1;

// Bullshark
```

```
if r % 2 != 0 || r < 2 {
return Ok(Vec::new());
}
let leader_round = r;
let support_round = round;
```

6.6.2 상태 관리 단순화

- **더 적은 라운드 추적**: 과거 라운드 참조 감소
- **메모리 효율성**: 더 적은 상태 정보 유지
- **디버깅 용이성**: 단순한 로직으로 문제 추적 쉬움

6.7 네트워크 효율성

6.7.1 메시지 복잡도

```
pub struct NetworkEfficiency {
// Bullshark는 더 적은 라운드로 동일한 결과 달성
messages_per_commit: usize, // Tusk 대비 감소
network_utilization: f64, // 더 효율적인 대역폭 사용
latency_reduction: Duration, // 평균 지연시간 단축
}
```

6.7.2 대역폭 최적화

- **조기 커밋**: 불필요한 메시지 교환 감소
- **효율적 전파**: 리더 정보의 빠른 전파
- **네트워크 부하 감소**: 전체적인 네트워크 트래픽 감소

6.8 실제 성능 비교

6.8.1 벤치마크 결과

```
// 성능 비교 메트릭
pub struct PerformanceComparison {
tusk_latency: Duration, // ~4 라운드
bullshark_latency: Duration, // ~2 라운드
tusk_throughput: u64, // 기준값
bullshark_throughput: u64, // ~2배 향상
memory_usage_reduction: f64, // 메모리 사용량 감소
cpu_efficiency: f64, // CPU 효율성 향상
}
```

6.8.2 실제 배포 고려사항

- **호환성**: Tusk와 동일한 인터페이스 제공
- **마이그레이션**: Tusk에서 Bullshark로 쉬운 전환
- **설정 최적화**: 네트워크 환경에 맞는 파라미터 조정

6.9 사용 시나리오

6.9.1 Bullshark 선택 기준

- **낮은 지연시간 요구**: 실시간 애플리케이션
- **높은 처리량 필요**: 대용량 트랜잭션 처리
- **부분 동기 환경**: 네트워크 지연이 가변적인 환경
- **리소스 효율성**: 제한된 컴퓨팅 리소스

6.9.2 Tusk 선택 기준

- **최대 안전성**: 극도로 안전한 환경 요구
- **검증된 안정성**: 장기간 검증된 알고리즘 선호
- **보수적 접근**: 새로운 기술 도입에 신중한 환경

Bullshark는 Tusk의 장점을 유지하면서 성능을 크게 개선한 차세대 컨센서스 프로토콜로, 현대적인 블록체인 시스템의 요구사항을 더 잘 만족시킵니다.